

MÓDULOS INTEGRADOS
GA8-220501096-AA1-EV02

Instructora:
DORIS MATILDE ORDOÑEZ LASSO

Aprendices:
DIEGO ANDRÉS JAIMES GELVEZ
DIEGO ARMANDO MOLANO GUTIERREZ
ERVIN ADOLFO OLAYA MURILLO
LEIDER ARBEY QUINTERO MOYA

TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE (ADSO)
CENTRO DE TELEINFORMÁTICA Y PRODUCCIÓN INDUSTRIAL
SERVICIO NACIONAL DE APRENDIZAJE (SENA)

FICHA: 3118301

2026

TABLA DE CONTENIDO

1. Introducción.....	3
2. Objetivos.....	4
2.1 <i>Objetivo General.</i>	4
2.2 <i>Objetivos Específicos.</i>	4
3. Alcance.....	5
4. Módulos Integrados	6
4.1 <i>Módulo de Autenticación.</i>	6
4.2 <i>Módulo de Gestión de Usuarios.</i>	8
4.3 <i>Módulo de gestión de vehículos.</i>	11
4.4 <i>Módulo de gestión de parqueadero.</i>	14
4.5 <i>Módulo de facturación.</i>	17
4.6 <i>Módulo de reportes.</i>	19
4.7 <i>Módulo de gestión de clientes.</i>	21
4.8 <i>Módulo de gestión de tickets.</i>	22
5. Conclusiones	24
6. Enlace GitHub de repositorio	25

1. Introducción

El desarrollo de software moderno exige la implementación de metodologías que permitan construir aplicaciones robustas, escalables y de fácil mantenimiento. Dentro de estas metodologías, el desarrollo modular se ha consolidado como una práctica fundamental, ya que permite dividir un sistema en componentes independientes que pueden ser diseñados, desarrollados y probados de manera separada. Esta estrategia facilita la organización del trabajo, optimiza los recursos disponibles y mejora la calidad del producto final.

La integración de módulos constituye una etapa crítica dentro del ciclo de vida del desarrollo de software. Una vez que los componentes han sido desarrollados individualmente, es necesario garantizar que interactúen de manera adecuada para conformar un sistema funcional y cohesionado. Este proceso requiere una adecuada planificación, documentación y ejecución de pruebas que permitan validar el cumplimiento de los requerimientos establecidos.

En el contexto de la formación en Análisis y Desarrollo de Software, la entrega de módulos integrados representa una evidencia fundamental para demostrar la aplicación de conocimientos técnicos y metodológicos adquiridos durante el proceso de aprendizaje. Dicha entrega no solo contempla el código fuente del sistema, sino también la documentación técnica, los manuales, los ejecutables y las evidencias de pruebas realizadas.

Asimismo, resulta indispensable gestionar el desarrollo mediante herramientas de control de versiones como Git o SVN, las cuales permiten mantener un historial de cambios, facilitar el trabajo colaborativo y garantizar la trazabilidad del proyecto. De igual manera, la documentación de los ambientes de desarrollo, pruebas, servidores y bases de datos contribuye a asegurar la correcta implementación y mantenimiento del sistema.

El presente documento tiene como finalidad describir los elementos necesarios para la entrega de módulos integrados de un sistema de software, abordando los requerimientos, la documentación asociada, las pruebas realizadas y los aspectos técnicos que garantizan la calidad, funcionalidad y confiabilidad de la solución desarrollada.

2. Objetivos

2.1 Objetivo General.

Presentar de manera organizada y documentada la entrega de los módulos integrados del sistema de software desarrollado, garantizando el cumplimiento de los requerimientos funcionales y técnicos establecidos, así como la evidencia de las pruebas, configuraciones y documentación necesarias para su correcta implementación y operación.

2.2 Objetivos Específicos.

- Identificar y validar los requerimientos funcionales y no funcionales aprobados para el desarrollo del sistema.
- Entregar el código fuente del proyecto mediante un repositorio de control de versiones que garantice la trazabilidad de los cambios realizados.
- Documentar los módulos y componentes del sistema, especificando las entradas, procesos y salidas correspondientes.
- Presentar las evidencias de las pruebas ejecutadas sobre cada módulo y sus respectivos resultados.
- Proporcionar los manuales técnicos, archivos ejecutables y direcciones de despliegue necesarias para la instalación, administración y uso del sistema.

3. Alcance

El presente documento comprende la entrega integral de los módulos desarrollados para el sistema de software, incluye los componentes funcionales que conforman la solución tecnológica propuesta. Se busca evidenciar que cada módulo cumple con los requerimientos previamente definidos y aprobados por las partes interesadas.

Dentro del alcance se incluye la presentación del código fuente almacenado en un repositorio de control de versiones, permitiendo la revisión, seguimiento y mantenimiento del proyecto. Este repositorio constituye una evidencia de las actividades realizadas durante el desarrollo y facilita futuras actualizaciones del sistema.

También se contempla la entrega de los archivos ejecutables y de las URL correspondientes a los entornos donde los módulos han sido desplegados. Esto permite verificar el correcto funcionamiento de la aplicación y facilita su acceso para procesos de evaluación, validación y puesta en producción.

Adicionalmente, se incluyen los documentos técnicos asociados a cada módulo y componente, detallando la información relacionada con datos de entrada, procesamiento y salida. Esta documentación sirve como referencia para desarrolladores, administradores y demás usuarios técnicos involucrados en el mantenimiento del sistema.

Finalmente, el alcance considera la documentación de las pruebas realizadas, las configuraciones de servidores y bases de datos, los ambientes de desarrollo y pruebas, así como los manuales técnicos necesarios para garantizar la correcta instalación, operación y soporte de la solución tecnológica entregada.

4. Módulos Integrados

El Sistema Inteligente de Gestión de Parquadero fue desarrollado bajo una arquitectura modular que permite la separación de responsabilidades, facilitando el mantenimiento, la escalabilidad y la reutilización del código. A continuación, se describen los módulos implementados.

4.1 Módulo de Autenticación.

El módulo de autenticación es el encargado de controlar el acceso seguro al sistema mediante la validación de credenciales de los usuarios registrados. Su principal objetivo es garantizar que únicamente los usuarios autorizados puedan acceder a las funcionalidades disponibles según su rol dentro de la aplicación.

Este módulo implementa mecanismos de seguridad basados en JSON Web Tokens (JWT), permitiendo la identificación y autenticación de usuarios durante cada solicitud realizada al servidor. Además, protege las rutas sensibles de la aplicación mediante middleware de validación, evitando accesos no autorizados y fortaleciendo la seguridad de la información almacenada en el sistema.

El funcionamiento de este módulo constituye la base de la seguridad de toda la plataforma, ya que administra la creación de sesiones, la validación de permisos y el control de acceso a los diferentes módulos del sistema.

Funcionalidades

- Inicio de sesión de usuarios.

- Validación de credenciales.
- Generación de tokens JWT.
- Protección de rutas privadas.
- Gestión de sesiones.

Componentes Implementados

- auth.routes.js
- auth.controller.js
- auth.service.js
- auth.middleware.js
- jwt.js

Entradas

- Correo electrónico.
- Contraseña.

Salidas

- Token JWT.
- Información del usuario autenticado.
- Mensajes de validación.

```

/**
 * Servicio de autenticacion.
 */
const bcrypt = require('bcrypt')
const AppError = require('../utils/AppError')
const userRepository = require('../repositories/user.repository')
const userService = require('../user.service')
const { mapUser } = require('../utils/mapper')
const { signAccessToken, signRefreshToken, verifyToken } = require('../utils/jwt')

const buildSession = (user) => ({
  user: mapUser(user),
  token: signAccessToken(user),
  refreshToken: signRefreshToken(user),
  expiresAt: Date.now() + 1000 * 60 * 60,
})

const register = async (payload) => userService.create({ ...payload, rol: 'CLIENTE' }, 'CLIENTE')

const login = async (payload) => {
  const correo = payload.correo || payload.email
  const user = await userRepository.findByEmailWithPassword(correo)
  if (!user || user.estado !== 'Activo') throw new AppError('Credenciales invalidas', 401)
  const valid = await bcrypt.compare(payload.password, user.password)
  if (!valid) throw new AppError('Credenciales invalidas', 401)
  return buildSession(user)
}

const refresh = async (refreshToken) => {
  const decoded = verifyToken(refreshToken)
  if (decoded.type !== 'refresh') throw new AppError('Refresh token invalido', 401)
  const user = await userRepository.findById(decoded.id)
  if (!user) throw new AppError('Usuario no encontrado', 404)
  return buildSession(user)
}

module.exports = { register, login, refresh }

```

4.2 Módulo de Gestión de Usuarios.

El módulo de gestión de usuarios permite administrar toda la información relacionada con las personas que interactúan con el sistema. Su finalidad es facilitar el registro, consulta, actualización y eliminación de usuarios, garantizando una adecuada administración de los perfiles existentes.

A través de este módulo se controla la asignación de roles y permisos, permitiendo diferenciar las funcionalidades disponibles para administradores, operadores y demás tipos de usuarios definidos en la aplicación. Asimismo, facilita la gestión de datos personales y credenciales de acceso necesarias para el funcionamiento del sistema.

La correcta administración de usuarios permite mantener la integridad de la información y establecer controles adecuados sobre las operaciones realizadas dentro de la plataforma.

Funcionalidades

- Crear usuarios.
- Consultar usuarios.
- Actualizar información.
- Eliminar usuarios.
- Asignar roles.

Componentes Implementados

- user.routes.js
- user.controller.js
- user.service.js
- user.repository.js
- roles.js

Entradas

- Nombre.
- Correo electrónico.

- Contraseña.
- Rol.

Salidas

- Registro de usuarios.
- Listado de usuarios.
- Confirmaciones de actualización y eliminación.

```
/**
 * Servicio de usuarios y clientes.
 */
const bcrypt = require('bcrypt')
const AppError = require('../utils/AppError')
const userRepository = require('../repositories/user.repository')
const { mapUser } = require('../utils/mapper')

const normalizeUserPayload = (payload, defaultRole = 'CLIENTE') => ({
  nombre: payload.nombre || payload.name?.split(' ')[0],
  apellido: payload.apellido || payload.name?.split(' ').slice(1).join(' ') || '',
  documento: payload.documento,
  correo: payload.correo || payload.email,
  telefono: payload.telefono,
  rol: payload.rol || payload.role || defaultRole,
  estado: payload.estado || 'Activo',
})

const list = async (filters = {}) => (await userRepository.findAll(filters)).map(mapUser)

const getById = async (id) => {
  const user = await userRepository.findById(id)
  if (!user) throw new AppError('Usuario no encontrado', 404)
  return mapUser(user)
}

const create = async (payload, defaultRole) => {
  const data = normalizeUserPayload(payload, defaultRole)
  if (!data.correo) throw new AppError('El correo es obligatorio', 400)
  const exists = await userRepository.findByEmailWithPassword(data.correo)
  if (exists) throw new AppError('Ya existe un usuario con ese correo', 409)
  const hashedPassword = await bcrypt.hash(payload.password || '123456', 10)
  const id = await userRepository.create({ ...data, password: hashedPassword })
  return getById(id)
}

const update = async (id, payload) => {
  await getById(id)
  const data = normalizeUserPayload(payload)
  const columns = {
    nombre: data.nombre,
    apellido: data.apellido,
    documento: data.documento,
    correo: data.correo,
    telefono: data.telefono,
    rol: payload.rol || payload.role,
    estado: payload.estado,
  }
}
```

```

    if (payload.password) columns.password = await bcrypt.hash(payload.password, 10)
    await userRepository.update(id, columns)
    return getById(id)
  }

  ✓ const remove = async (id) => {
    await getById(id)
    await userRepository.remove(id)
  }

  module.exports = { list, getById, create, update, remove }

```

4.3 Módulo de gestión de vehículos.

Este módulo tiene como propósito administrar toda la información asociada a los vehículos registrados dentro del sistema de parqueadero. Permite almacenar y gestionar datos relevantes como placa, marca, modelo, color y propietario del vehículo.

Su implementación facilita la consulta rápida de información vehicular, permitiendo mantener un control organizado de los automóviles que ingresan al parqueadero. Además, ofrece funcionalidades para modificar registros existentes o eliminarlos cuando sea necesario.

La gestión eficiente de vehículos es fundamental para garantizar la trazabilidad de los movimientos realizados dentro del parqueadero y para apoyar otros procesos como el control de acceso, la facturación y la generación de reportes.

Funcionalidades

- Registrar vehículos.
- Consultar vehículos.
- Actualizar información.
- Eliminar vehículos.
- Búsqueda de vehículos.

Componentes Implementados

- vehicle.routes.js
- vehicle.controller.js
- vehicle.service.js
- vehicle.repository.js

Entradas

- Placa.
- Marca.
- Modelo.
- Color.
- Propietario.

Salidas

- Vehículo registrado.
- Listado de vehículos.
- Confirmación de actualización.
- Confirmación de eliminación.

```

/**
 * Servicio de vehiculos.
 */
const AppError = require('../utils/AppError')
const vehicleRepository = require('../repositories/vehicle.repository')
const userRepository = require('../repositories/user.repository')
const { mapVehicle } = require('../utils/mapper')

/**
 * Busca el cliente por ID o por nombre cuando el frontend envia texto.
 *
 * @param {Object} payload Datos recibidos.
 * @returns {Promise<number|null>} ID del cliente o null.
 */
const resolveClientId = async (payload) => {
  if (payload.clienteId) return payload.clienteId
  if (!payload.cliente) return null
  const matches = await userRepository.findAll({ rol: 'CLIENTE', q: payload.cliente })
  return matches[0]?.id || null
}

const normalize = async (payload) => ({
  placa: payload.placa,
  tipoVehiculo: payload.tipoVehiculo || payload.tipo || 'Carro',
  marca: payload.marca,
  modelo: payload.modelo,
  color: payload.color,
  clienteId: await resolveClientId(payload),
  estado: payload.estado,
})

const list = async (filters) => (await vehicleRepository.findAll(filters)).map(mapVehicle)

const getById = async (id) => {
  const vehicle = await vehicleRepository.findById(id)
  if (!vehicle) throw new AppError('Vehiculo no encontrado', 404)
  return mapVehicle(vehicle)
}

const getByPlate = async (placa) => {
  const vehicle = await vehicleRepository.findByPlate(placa)
  if (!vehicle) throw new AppError('Vehiculo no encontrado', 404)
  return mapVehicle(vehicle)
}

```

```

    ✓ const create = async (payload) => {
      const data = await normalize(payload)
      const exists = await vehicleRepository.findByPlate(data.placa)
      if (exists) throw new AppError('Ya existe un vehiculo con esa placa', 409)
      const id = await vehicleRepository.create(data)
      return getById(id)
    }

    ✓ const update = async (id, payload) => {
      await getById(id)
      const data = await normalize(payload)
      ✓ const columns = {
        placa: data.placa?.toUpperCase(),
        tipo_vehiculo: data.tipoVehiculo,
        marca: data.marca,
        modelo: data.modelo,
        color: data.color,
        cliente_id: data.clienteId,
        estado: data.estado,
      }
      await vehicleRepository.update(id, columns)
      return getById(id)
    }

    ✓ const remove = async (id) => {
      await getById(id)
      await vehicleRepository.remove(id)
    }

    module.exports = { list, getById, getByPlate, create, update, remove }

```

4.4 Módulo de gestión de parqueadero.

El módulo de parqueadero constituye el núcleo operativo del sistema, ya que permite administrar los movimientos de ingreso y salida de vehículos dentro de las instalaciones. Su objetivo principal es mantener un registro actualizado de la ocupación y disponibilidad de espacios de estacionamiento.

Este módulo registra la fecha y hora de entrada de cada vehículo, así como la fecha y hora de salida cuando abandona el parqueadero. Gracias a esta información es posible calcular el

tiempo de permanencia, generar cobros asociados y llevar un control histórico de las operaciones realizadas.

Además, proporciona información en tiempo real sobre la capacidad utilizada del parqueadero, facilitando la toma de decisiones administrativas y mejorando la eficiencia operativa del servicio.

Funcionalidades

- Registro de ingreso.
- Registro de salida.
- Control de ocupación.
- Consulta de movimientos.

Componentes Implementados

- parking.routes.js
- parking.controller.js
- parking.service.js
- parking.repository.js

Entradas

- Placa del vehículo.
- Fecha y hora de ingreso.
- Fecha y hora de salida.

Salidas

- Registro de movimiento.

- Estado del parqueadero.

```

1  /**
2  | * Servicio de parqueadero: ingresos, salidas, disponibilidad y sedes.
3  | */
4  const AppError = require('../utils/AppError')
5  const parkingRepository = require('../repositories/parking.repository')
6  const vehicleRepository = require('../repositories/vehicle.repository')
7  const userRepository = require('../repositories/user.repository')
8  const { mapTicket } = require('../utils/mapper')
9
10 const calculatePayment = (ticket) => {
11   const startedAt = new Date(ticket.fechaIngreso)
12   const endedAt = new Date()
13   const hours = Math.max(1, Math.ceil((endedAt - startedAt) / (1000 * 60 * 60)))
14   const rate = ticket.placa && ticket.placa.length === 6 ? Number(process.env.TARIFA_HORA_CARRO || 5000)
15   : Number(process.env.TARIFA_HORA_MOTOCICLETA || 3000)
16   return { tiempoTotal: `${hours} hora(s)`, valorPagado: hours * rate }
17 }
18
19 const listTickets = async (filters) => (await parkingRepository.findTickets(filters)).map(mapTicket)
20
21 const registerEntry = async (payload) => {
22   let vehicle = await vehicleRepository.findByPlate(payload.placa)
23   if (!vehicle) {
24     const clients = payload.cliente ? await userRepository.findAll({ rol: 'CLIENTE', q: payload.cliente }) : []
25     const id = await vehicleRepository.create({
26       placa: payload.placa,
27       tipoVehiculo: payload.tipoVehiculo || payload.tipo || 'Carro',
28       marca: payload.marca,
29       clienteId: clients[0]?.id || null,
30       estado: 'En parqueadero',
31     })
32     vehicle = await vehicleRepository.findById(id)
33   }
34
35   const active = await parkingRepository.findActiveByVehicleId(vehicle.id)
36   if (active) throw new AppError('El vehículo ya tiene un ingreso activo', 409)
37
38   const id = await parkingRepository.createTicket({
39     vehiculoId: vehicle.id,
40     espacioAsignado: payload.espacioAsignado || payload.espacio || 'Sin asignar',
41   })
42   await vehicleRepository.update(vehicle.id, { estado: 'En parqueadero' })
43   return mapTicket(await parkingRepository.findTicketById(id))
44 }
45

```

```

46  const registerExit = async (id, payload = {}) => {
47    const ticket = await parkingRepository.findTicketById(id)
48    if (!ticket) throw new AppError('Ticket no encontrado', 404)
49    if (ticket.estado !== 'Activo') throw new AppError('El ticket ya esta cerrado', 409)
50    const calculated = calculatePayment(ticket)
51    await parkingRepository.closeTicket(id, {
52      tiempoTotal: payload.tiempoTotal || calculated.tiempoTotal,
53      valorPagado: payload.valorPagado || calculated.valorPagado,
54    })
55    await vehicleRepository.update(ticket.vehiculoId, { estado: 'Disponible' })
56    return mapTicket(await parkingRepository.findTicketById(id))
57  }
58
59  const listSedes = async () => parkingRepository.listSedes()
60
61  const createSede = async (payload) => {
62    const id = await parkingRepository.createSede(payload)
63    return (await parkingRepository.listSedes()).find((item) => item.id === id)
64  }
65
66  const availability = async () => parkingRepository.availability()
67
68  module.exports = { listTickets, registerEntry, registerExit, listSedes, createSede, availability }
69

```

4.5 Módulo de facturación.

El módulo de facturación se encarga de gestionar los procesos relacionados con el cálculo y generación de cobros por el uso del servicio de parqueadero. Su función principal consiste en determinar el valor que debe cancelar cada cliente en función del tiempo de permanencia de su vehículo y las tarifas establecidas por la administración.

Este módulo automatiza los cálculos de pago, reduciendo errores operativos y garantizando la aplicación consistente de las políticas tarifarias definidas. Asimismo, permite almacenar el historial de facturación, facilitando el seguimiento de los ingresos generados por el parqueadero.

La integración con los módulos de vehículos y parqueadero permite obtener información precisa para emitir facturas de manera eficiente y transparente.

Funcionalidades

- Cálculo automático de tarifas.
- Generación de facturas.
- Consulta de facturas.
- Historial de pagos.

Componentes Implementados

- `billing.routes.js`
- `billing.controller.js`
- `billing.service.js`
- `billing.repository.js`

Entradas

- Tiempo de permanencia.
- Tarifa establecida.
- Datos del vehículo.

Salidas

- Valor a pagar.
- Factura generada.
- Historial de facturación.

```

1  /**
2   * Servicio de facturas y pagos.
3   */
4   const billingRepository = require('../repositories/billing.repository')
5
6   const listInvoices = async () => billingRepository.listInvoices()
7   const getInvoice = async (id) => billingRepository.findInvoiceById(id)
8   const createInvoice = async (payload) => {
9     const id = await billingRepository.createInvoice(payload)
10    return billingRepository.findInvoiceById(id)
11  }
12
13  const listPayments = async () => billingRepository.listPayments()
14  const createPayment = async (payload) => {
15    const id = await billingRepository.createPayment(payload)
16    return { id: `PAY-${id}`, estado: payload.estado || 'Aprobado', ...payload }
17  }
18
19  module.exports = { listInvoices, getInvoice, createInvoice, listPayments, createPayment }
20

```

4.6 Módulo de reportes.

El módulo de reportes proporciona herramientas para la consulta y análisis de la información almacenada en el sistema. Su finalidad es apoyar los procesos administrativos mediante la generación de reportes que permitan evaluar el comportamiento operativo y financiero del parqueadero.

Los reportes pueden incluir información sobre ingresos y salidas de vehículos, niveles de ocupación, facturación generada, estadísticas de uso y otros indicadores relevantes para la gestión del negocio.

Gracias a este módulo, los administradores pueden obtener información consolidada que facilita la toma de decisiones, la identificación de tendencias y la evaluación del desempeño general del sistema.

Funcionalidades

- Reportes de ingresos.

- Reportes de salidas.
- Reportes de facturación.
- Reportes estadísticos.

Componentes Implementados

- report.routes.js
- report.controller.js
- report.service.js

Entradas

- Rango de fechas.
- Tipo de reporte.

Salidas

- Reportes detallados.
- Estadísticas de operación.

```

1  /**
2   * Servicio de reportes resumidos para dashboards.
3   */
4   const parkingRepository = require('../repositories/parking.repository')
5   const billingRepository = require('../repositories/billing.repository')
6
7   const dashboard = async () => {
8     const [availability, tickets, facturas] = await Promise.all([
9       parkingRepository.availability(),
10      parkingRepository.findTickets(),
11      billingRepository.listInvoices(),
12    ])
13    const ingresosHoy = facturas.reduce((sum, invoice) => sum + Number(invoice.total || 0), 0)
14    return {
15      metricas: [
16        { label: 'Ingresos hoy', value: ingresosHoy, trend: '+0%' },
17        { label: 'Ocupacion', value: `${availability.total ? Math.round((availability.ocupados
18          / availability.total) * 100) : 0}%`, trend: '+0%' },
19        { label: 'Tickets activos', value: tickets.filter(ticket => ticket.estado === 'Activo').length, trend: '+0' },
20        { label: 'Espacios libres', value: availability.disponibles || 0, trend: '+0' },
21      ],
22      ingresos: [
23        { dia: 'Lun', valor: ingresosHoy },
24        { dia: 'Mar', valor: ingresosHoy },
25        { dia: 'Mie', valor: ingresosHoy },
26        { dia: 'Jue', valor: ingresosHoy },
27        { dia: 'Vie', valor: ingresosHoy },
28      ],
29      ocupacion: [
30        { name: 'Ocupados', value: availability.ocupados || 0 },
31        { name: 'Disponibles', value: availability.disponibles || 0 },
32      ],
33      facturas,
34    }
35  }
36
37  module.exports = { dashboard }
38

```

4.7 Módulo de gestión de clientes.

El módulo de gestión de clientes permite administrar la información de las personas o entidades que utilizan los servicios del parqueadero. Su objetivo es mantener una base de datos organizada y actualizada con los datos de contacto y la información necesaria para la identificación de los clientes.

Este módulo facilita la relación entre los clientes y los vehículos registrados, permitiendo llevar un control más eficiente de las operaciones realizadas y mejorando la calidad del servicio prestado.

La centralización de la información de clientes contribuye a optimizar los procesos administrativos y a mejorar la experiencia de los usuarios del sistema.

Funcionalidades

- Registro de clientes.
- Consulta de clientes.
- Actualización de información.
- Eliminación de clientes.

Componentes Implementados

- `client.routes.js`

Entradas

- Datos personales del cliente.
- Información de contacto.

Salidas

- Registro actualizado de clientes.

4.8 Módulo de gestión de tickets.

El módulo de tickets es responsable de generar y administrar los comprobantes asociados a los movimientos de ingreso y salida de vehículos. Cada ticket actúa como un identificador único que permite relacionar la información del vehículo, el cliente, los tiempos de permanencia y los procesos de facturación.

La utilización de tickets facilita el seguimiento detallado de cada operación realizada dentro del parqueadero, garantizando la trazabilidad de los registros y permitiendo verificar la información cuando sea necesario.

Este módulo también sirve como mecanismo de apoyo para los procesos de control interno, auditoría y generación de reportes, proporcionando un historial confiable de las actividades desarrolladas dentro del sistema.

Funcionalidades

- Generación de tickets.
- Consulta de tickets.
- Asociación con facturación.
- Control de movimientos.

Componentes Implementados

- ticket.routes.js

Entradas

- Datos del vehículo.
- Fecha y hora de ingreso.

Salidas

- Ticket generado.
- Historial de tickets.

5. Conclusiones

La integración de módulos constituye una práctica esencial en el desarrollo de software moderno, ya que permite construir sistemas complejos de manera organizada, facilitando la escalabilidad, mantenibilidad y reutilización de componentes.

La correcta documentación de los requerimientos, componentes y configuraciones técnicas contribuye significativamente a la calidad del producto final, permitiendo una mejor comprensión del sistema y facilitando futuros procesos de actualización y mantenimiento.

El uso de herramientas de control de versiones proporciona trazabilidad sobre los cambios realizados durante el desarrollo, favoreciendo el trabajo colaborativo y reduciendo los riesgos asociados a la gestión del código fuente.

Las pruebas ejecutadas sobre cada módulo integrado permiten validar el cumplimiento de los requerimientos establecidos, garantizando que la interacción entre los componentes se realice de manera adecuada y asegurando la estabilidad del sistema.

La entrega organizada de ejecutables, documentación técnica, manuales y evidencias de despliegue constituye una buena práctica profesional que fortalece la gestión del proyecto y facilita la adopción, operación y sostenibilidad de la solución tecnológica desarrollada.

6. Enlace GitHub de repositorio

<https://github.com/adolfoolaya-code/GA8-220501096-AA1-EV02-m-dulos-integrados/>